CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 04
Hashing & Randomized Data Structures

"$O(1)$ expected" and "$O(\log n)$ expected": expected over what, and what happens when someone chooses your keys?

Saturday 17 October 2026 · 3 hours

Due today: PS1 · Project M1 (team + topic)

Where we are

L03 gave us

- Expected time $= \max_x \mathbb{E}_r[T(x, r)]$: worst-case input, our coins
- Guarantees hold against an **oblivious** adversary
- Indicators + linearity; Markov; amplification

TODAY

Apply the same idea to **data structures**: choose the hash function (or the tower heights) at random, so that no fixed set of keys is bad. Then compare with deterministic worst-case structures.

Motivating incident: hash flooding (2011)

At the 28C3 conference (Dec 2011), Klink and Wälde showed that many web platforms (PHP, Java, Python, Ruby, ASP.NET…) stored HTTP POST parameters in hash tables with a **fixed, public** hash function.

- An attacker sends one request with n parameter names that all collide.
- Each insert scans the whole chain: $\Theta(n^2)$ total.
- A ~2 MB request could keep a CPU core busy for tens of seconds or more.

Fixes shipped: randomized (seeded) hash functions, and limits on the number of parameters.

DISCUSS · 3 MIN

The hash tables were "$O(1)$ expected". Which assumption behind that claim did the attacker violate? What exactly must a fix randomize?

BLOCK A · 00:10–01:00

Universal hashing

From "assume random keys" to "choose a random function"

Every fixed hash function has a bad input

Universe U of keys, table size m, fixed $h : U \to \{0, \ldots, m-1\}$.

> **PIGEONHOLE**
> Some slot receives at least $|U|/m$ keys. If $|U| \geq nm$, there are n keys that **all** hash to the same slot.

For $h(k) = k \bmod 2^{16}$: all multiples of 2^{16} (aligned addresses, IDs with low bits zero, timestamps rounded to seconds…).

Two ways to get "expected $O(1)$":

	Randomness	Holds for
Simple uniform hashing (CLRS 11.2)	assumed in the **keys**	"typical" key sets only
Universal hashing	**in the choice of h**	**every key set fixed in advance**

Universal families

DEFINITION (CARTER & WEGMAN 1979)

A family $\mathcal{H}$ of functions $U \to \{0, \dots, m-1\}$ is **universal** if for all $x \neq y \in U$:

$$\Pr_{h \in \mathcal{H}}\big[h(x) = h(y)\big] \leq \frac{1}{m}.$$

(Probability over a uniformly random choice of h; the keys are *fixed*.)

THEOREM

Chaining, h drawn from a universal family, any fixed set S of n keys, load factor $\alpha = n/m$. For any key x, the expected number of keys of $S \setminus \{x\}$ in x's chain is $\leq \alpha$. So a search costs expected $O(1+\alpha)$.

PROOF

Let $C_y = 1[h(y) = h(x)]$. The chain length is $\sum_{y \in S \setminus \{x\}} C_y$, and by linearity $\mathbb{E} = \sum_y \Pr[h(y) = h(x)] \leq \frac{n}{m}$. ∎

A universal family and why it works

Pick a prime $p >$ every key. For $a \in \{1, \ldots, p-1\}$, $b \in \{0, \ldots, p-1\}$:

$$h_{a,b}(k) = \big((ak + b) \bmod p\big) \bmod m.$$

Proof idea. Fix $x \neq y$. Let $r = (ax + b) \bmod p$ and $s = (ay + b) \bmod p$.

1. $r \neq s$, since $r - s \equiv a(x - y) \not\equiv 0$ (p is prime).
2. The map $(a, b) \mapsto (r, s)$ is a **bijection** onto pairs with $r \neq s$: solve $a = (r - s)(x - y)^{-1}$, $b = r - ax$.
3. So (r, s) is uniform over the $p(p-1)$ pairs with $r \neq s$. For each r, at most $\lceil p/m \rceil - 1 \leq \frac{p-1}{m}$ values $s \neq r$ have $s \equiv r \pmod m$.

Hence $\Pr[\text{collision}] \leq \frac{p(p-1)/m}{p(p-1)} = \frac{1}{m}$.

IN PRACTICE: MULTIPLY-SHIFT

$m = 2^\ell$, w-bit keys, random **odd** a:

$$h_a(k) = \big(a \cdot k \bmod 2^w\big) \gg (w - \ell).$$

One multiplication and one shift. $\Pr[\text{collision}] \leq \frac{2}{m}$ (Dietzfelbinger et al. 1997): "universal up to a factor 2". It is used in today's experiment.

Integer division by a prime is slow, which is why fast hash tables use multiply-shift or tabulation hashing. Strings need polynomial hashing (L11).

Open addressing: more independence needed

- R4: unsuccessful search in open addressing costs $\leq \frac{1}{1-\alpha}$ expected probes under **uniform hashing**, an idealised assumption (every probe sequence is a random permutation).
- For **linear probing** with a real hash family, pairwise universality is *not* enough: some 2-universal families give $\Theta(\log n)$ expected probes.
- **5-independent** families give expected $O(1)$ (Pagh, Pagh, Ružić 2007); simple tabulation hashing also works (Pătraşcu & Thorup 2012).

TAKE-AWAY

"Expected $O(1)$" depends on **which** randomness assumption: fully random, k-independent, universal. Chaining needs only universal hashing. Linear probing needs more. The worst case is $\Theta(n)$ for all of them.

Linear probing is often faster in practice (cache locality). The theory explains what it requires.

Exercise A: attack and defend

1. A table uses $m = 1000$ and $h(k) = k \bmod 1000$. Give 100 distinct keys that all collide. Then give a key set that looks "natural" but still causes heavy collisions.
2. With $h_{a,b}$ ($p = 10007$, $m = 1000$), compute the expected number of **colliding pairs** among $n = 1000$ fixed keys. What is the expected number of keys sharing a slot with a given key x?
3. A developer picks a, b at random **once** and hard-codes them into the open-source library. Is the table still protected? Against whom?
4. Why is $h(k) = (ak \bmod p) \bmod m$ with **fixed** a not universal, even though it "looks random"?

Pairs · 8 min

BLOCK B · 01:10–02:00

Perfect hashing and skip lists

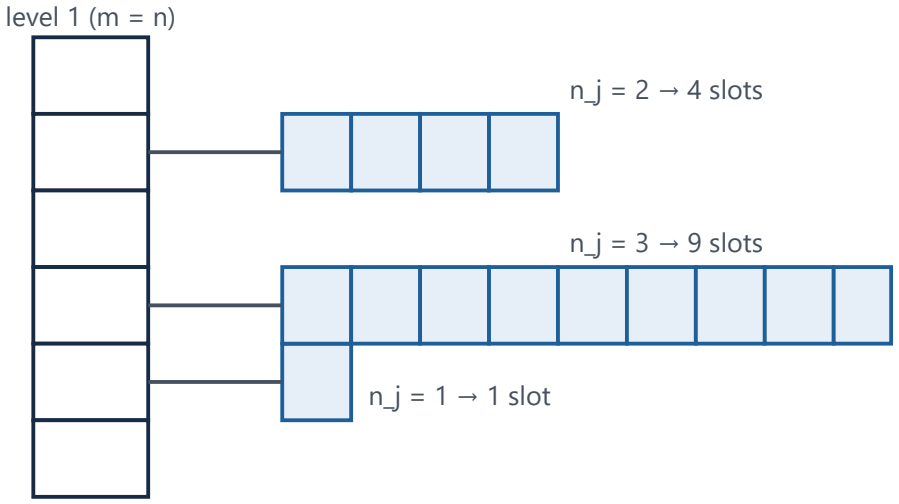
Worst-case $O(1)$ for static sets · randomized balancing for ordered sets

Perfect hashing (Fredman, Komlós, Szemerédi 1984)

Static set of n keys: build once, then only lookups.

1. **Level 1:** hash into $m = n$ buckets with a universal h. Bucket j gets n_j keys. Retry while $\sum_j n_j^2 > 4n$.
2. **Level 2:** bucket j gets its own table of size n_j^2 with its own universal h_j. Retry until it has **no collisions**.

Lookup: two hash evaluations, one comparison: **worst-case** $O(1)$.



each level-2 table: own random h_j, no collisions

Why FKS works: two expectations

LEVEL 2: A TABLE OF SIZE n_j^2 IS COLLISION-FREE WITH PROBABILITY $\geq \frac{1}{2}$

Expected colliding pairs $= \binom{n_j}{2} \cdot \frac{1}{n_j^2} < \frac{1}{2}$. By Markov, $\Pr[\geq 1 \text{ collision}] < \frac{1}{2}$. So the expected number of retries is < 2 (geometric).

■

LEVEL 1: TOTAL SPACE $\mathbb{E}\left[\sum_j n_j^2\right] < 2n$

$\sum_j n_j^2 = \sum_j \left(n_j + 2\binom{n_j}{2}\right) = n + 2 \cdot \#\{\text{colliding pairs}\}$, so

$$\mathbb{E}\Big[\sum_j n_j^2\Big] \leq n + 2\binom{n}{2}\frac{1}{n} < 2n.$$

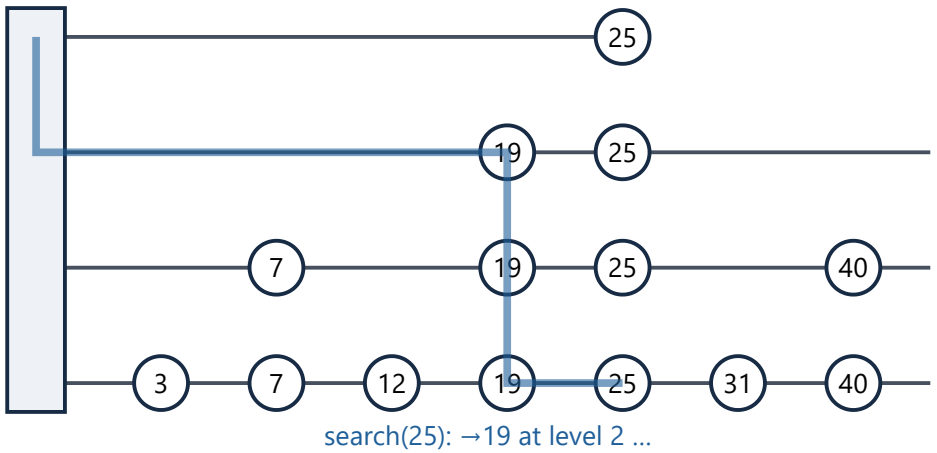
By Markov, $\Pr\left[\sum n_j^2 > 4n\right] < \frac{1}{2}$: expected < 2 level-1 attempts.

■

Result: expected $O(n)$ construction time, $O(n)$ space, worst-case $O(1)$ lookup. **Quadratic space per bucket is affordable because the buckets are small on average.**

Skip lists (Pugh 1990): randomized balancing

- Level 0: a sorted linked list of all n keys.
- Each key is promoted to the next level with probability $\frac{1}{2}$, independently: level L with $\Pr[L \geq h] = 2^{-h}$.
- **Search:** start at the top-left; move right while the next key is $< x$; otherwise drop down a level.
- **Insert:** search, flip coins for the height, splice into each level. **Delete:** unsplice. No rotations, no rebalancing.
- Expected pointers: $\sum_h n 2^{-h} = 2n$.



Skip-list analysis: expected $O(\log n)$, *not* worst case

HEIGHT

$H = \max_x L_x$. Union bound: $\Pr[H \geq h] \leq n2^{-h}$. So $\Pr[H \geq 3\log_2 n] \leq n^{-2}$ and $\mathbb{E}[H] \leq \log_2 n + 1$.

SEARCH COST (BACKWARD ANALYSIS)

Walk the search path **backwards** from the target. At each node, the path came from above with probability $\frac{1}{2}$ (the node was promoted) or from the left with probability $\frac{1}{2}$. Expected steps to climb one level $= 2$. Climbing $\leq H$ levels gives expected cost $\leq 2(\log_2 n + 1) + O(1)$.

WORST CASE: $\Theta(n)$

If the coins give every key level 0, a search walks the whole list. This happens with tiny probability, but not zero. The legacy slides' "average **and worst-case** $O(\log n)$" is **wrong**.

The guarantee is over **our coin flips** for any fixed operation sequence (oblivious adversary). An adversary who sees the heights (e.g. through timing) can choose which keys to query.

Deterministic vs. randomized balancing

	Red-black tree	Skip list	Treap (randomized BST)
Search/insert/delete	$O(\log n)$ **worst case**	$O(\log n)$ **expected**, w.h.p.	$O(\log n)$ expected
Restructuring per update	$O(1)$ rotations + recolouring	none (local pointer splices)	expected $O(1)$ rotations
Space	n nodes, 2 child pointers + colour	expected $2n$ forward pointers	n nodes + priority
Ordered ops (range, successor)	yes	yes (level-0 list)	yes
Concurrency	hard (rotations touch several nodes)	**easier**: lock-free designs exist (e.g. Java `ConcurrentSkipListMap`)	hard
Implementation effort	high (fix-up cases)	low	low

Exercise B: quick derivations

1. FKS: prove $\sum_j n_j^2 = n + 2 \cdot \#\{\text{colliding pairs}\}$.
2. Skip list with $p = \frac{1}{4}$: expected pointers per key? Expected height for $n = 10^6$?
3. Why does the skip-list analysis fail if keys choose their own heights deterministically, e.g. level = number of trailing zeros of the key?
4. You are offered a skip list with "worst-case $O(\log n)$ search". What would you ask the vendor?

Individually · 6 min

BLOCK C · 02:10–02:55

Hash table vs. tree vs. skip list, measured

Key distributions · adversarial keys · constants

Experiment 1: fixed vs. random hash function

Key set ($n = m = 2^{16}$)	Hash	max chain	avg probes	ms / 10^6 lookups
random 64-bit	$k \bmod m$	8	1.50	11.4
random 64-bit	multiply-shift	8	1.50	11.1
sequential $0..n-1$	$k \bmod m$	1	1.00	5.0
sequential $0..n-1$	multiply-shift	3	1.36	9.5
multiples of 2^{16}	$k \bmod m$	**65,536**	**32,772**	**9,218**
multiples of 2^{16}	multiply-shift	2	1.18	7.7

Chaining with vectors, load factor 1; `Student_Resources/hashing_skiplist_bench.cpp`, g++ 15.2 -O2.

INTERPRET · 4 MIN

1. Why is $k \bmod m$ *better* than random on sequential keys?

2. Predict the "avg probes" value 32,772 from theory.

3. Which rows could an average-case analysis have predicted, and which not?

Experiment 2: skip list vs. RB tree vs. hash table

$n = 10^6$, insert order	skip list ins	skip list find	std::map ins	std::map find	unordered_map ins	unordered_map find	SL height	SL forward steps / find
random	1924	1759	966	990	178	36	22	18.7
sorted	206	1302	138	746	121	29	22	18.9

Times in ms for 10^6 operations; lookups in random order; $\log_2 n = 19.9$. Straightforward skip list (per-node `std::vector`, $p = \frac{1}{2}$); `std::map` is a red-black tree.

THEORY CONFIRMED

Height 22 ≈ $\log_2 n + 2$; forward steps ≈ $\log_2 n$, independent of insertion order.

DISCUSS · 5 MIN

(1) Why is the skip list ~2× slower than `std::map` although both are $O(\log n)$? (2) Why is sorted insertion so much faster for both? (3) What would you need to know before choosing `unordered_map`?

Exercise C: choose the structure

1. A rate limiter keyed by client IP address; attackers control the source addresses; 1M active keys; p99 latency budget of 50 µs.
2. A game leaderboard: 10M players, score updates at 50k/s, queries "rank of player X" and "players ranked 1000–1100".
3. A spell checker's dictionary: 500k words, loaded once at startup, 10^8 lookups per day.
4. An in-memory ordered index shared by 32 threads with frequent inserts and range scans.

> **GROUPS · 12 MIN**
>
> For each: the structure, its guarantee **with type**, the assumption that guarantee needs, and what would change your choice.

Synthesis

1. Every fixed hash function has a bad key set (pigeonhole). "Expected $O(1)$" with a fixed h is an assumption about the keys.
2. **Universal hashing**: $\Pr_h[h(x) = h(y)] \leq \frac{1}{m} \Rightarrow$ expected chain $\leq 1 + \alpha$ for *every* fixed key set. Keep the seed secret.
3. **Perfect hashing**: static set, worst-case $O(1)$ lookups, expected $O(n)$ construction; $\mathbb{E} \sum n_j^2 < 2n$.
4. **Skip lists**: expected (and w.h.p.) $O(\log n)$; worst case $\Theta(n)$. Simple, ordered, concurrency-friendly.
5. Measured constants and memory layout often dominate between structures with the same $O(\cdot)$.

NEXT WEEK · L05 (SAT 24 OCT)

Augmented data structures (rank, select, intervals) and amortized analysis (aggregate, accounting, potential): what "$O(1)$ amortized" really promises.

Read: CLRS 4e Ch. 16–17; R3. **PS2** released next week; **M2** (problem + hypotheses) due next week.